```
#Simulated rewards for 3 steps
r1 = [1,1,-1]
r2 = [1,1,1]
R1,R2 = 0.0,0.0
#No bootstrapping
for i in range(len(r1)-1,0,-1):
    R1 = r1[i] + 0.99*R1
for i in range(len(r2)-1,0,-1):
    R2 = r2[i] + 0.99*R2
print("No bootstrapping")
print(R1,R2)
#With bootstrapping
R1,R2 = 1.0,1.0
for i in range(len(r1)-1,0,-1):
    R1 = r1[i] + 0.99*R1
for i in range(len(r2)-1,0,-1):
    R2 = r2[i] + 0.99*R2
print("With bootstrapping")
print(R1,R2)
>>> No bootstrapping
0.010000000000000009 1.99
With bootstrapping
0.9901 2.9701
```

To recap, in the plain policy gradient method of the previous chapter, we only trained a policy function that would output a probability distribution over all the actions, such that the predicted best action would be assigned the highest probability. Unlike Q-learning where a target value is learned, the policy function is directly reinforced to increase or decrease the probability of the action taken depending on the reward. Often the same action may produce opposite results in terms of reward, causing high variance in the training.

To mitigate this, we introduced a critic model (or in this chapter we used a single, two-headed model) that reduces the variance of the policy function updates by directly modeling the state value. This way, if the actor (policy) takes an action and gets an unusually big or small reward, the critic can moderate this big swing and prevent an unusually large (and possibly destructive) parameter update to the policy. This also leads to the notion of advantage, where instead of training the policy based on raw return (average accumulated rewards), we train based on how much better (or worse) the action was compared to what the critic predicted it would be. This is helpful, because if two actions both lead to the same positive